Penetration Testing Report

v.1.0

Ido.boker@walla.co.il

Ido Boker

Date:30/12/25

Executive Summary

Introduction

Penetration testing of Analyzer was carried out to evaluate the security level of the website. This comprehensive assessment, performed by ITSafe, involved a gray box security audit to assess the website's resilience against potential attacks and identify areas for enhancing data protection.

Scope

Web Application

The penetration testing was limited to:

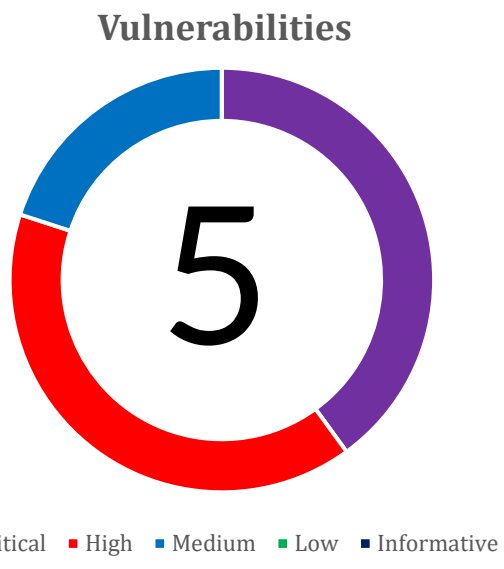
- 10.0.2.232

Penetration testing targeted the following functions:

- Tests against Advance Web Application Attacks.
- OWASP Top 10 possible vulnerabilities.

Conclusions

From our professional perspective, the overall security level of the system is **Low-Medium**. The application is vulnerable to XSS in the login form, SSRF via image inclusion, and RCE via the PING function in the admin panel. Also, the services are outdated and vulnerable to known CVE's



Identified Vulnerabilities

Item	*Test Type*	*Risk Level*	*Topic*	*General Explanation*	*Status*
1	**Applicative**	**Critical**	**RCE**	Remote Code Execution (RCE) is a critical vulnerability that allows an attacker to run malicious code on a target system from a remote location, without needing physical access.	**Vulnerable**
2	**Applicative**	**Critical**	**Known Vulnerabilities**	A known vulnerability is a publicly disclosed weakness or flaw in software, hardware, or an operating system that can be exploited by an attacker to compromise a system. These weaknesses can lead to unauthorized access, data breaches, or service disruptions.	**Vulnerable**
3	**Applicative**	**High**	**SSRF**	Server-Side Request Forgery (SSRF) is a vulnerability that allows an attacker to manipulate a server-side application into making unintended network requests to an arbitrary destination	**Vulnerable**
4	**Applicative**	**High**	**XSS**	Cross-Site Scripting (XSS) is a vulnerability that allows an attacker to inject malicious client-side scripts into legitimate websites. When a victim visits the compromised page, their browser executes the malicious code, which the browser trusts because it appears to originate from the legitimate, trusted website.	**Vulnerable**
5	**Applicative**	**Medium**	**Information Disclosure**	Information disclosure refers to the act of revealing or making accessible sensitive, private, or confidential information to individuals or entities that are not authorized to have access to it.	**Vulnerable**

Finding Details

1 RCE

Severity | Critical **Probability | Critical**

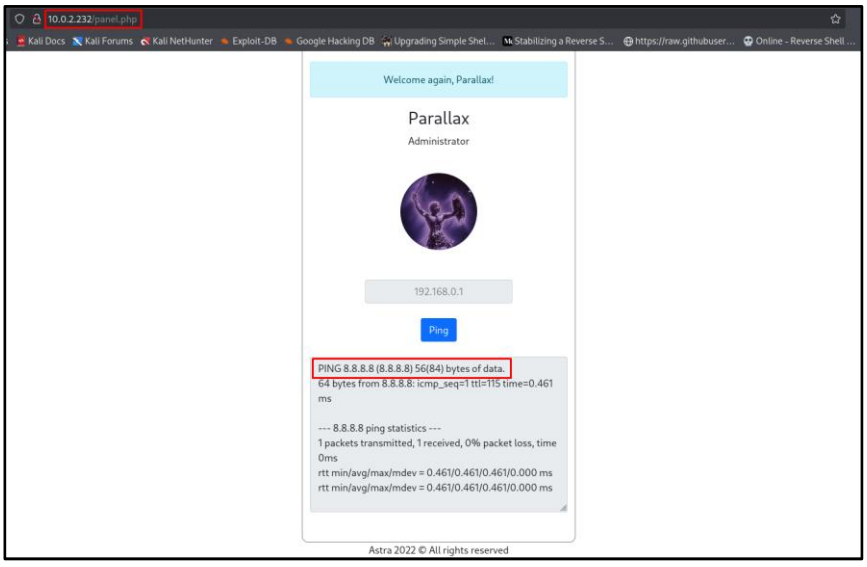
Vulnerability Description

Remote Code Execution (RCE) is a high-severity security vulnerability that allows an attacker to execute arbitrary commands or malicious code on a remote machine over a network. It is considered one of the most dangerous types of exploits because it can lead to a total system takeover.

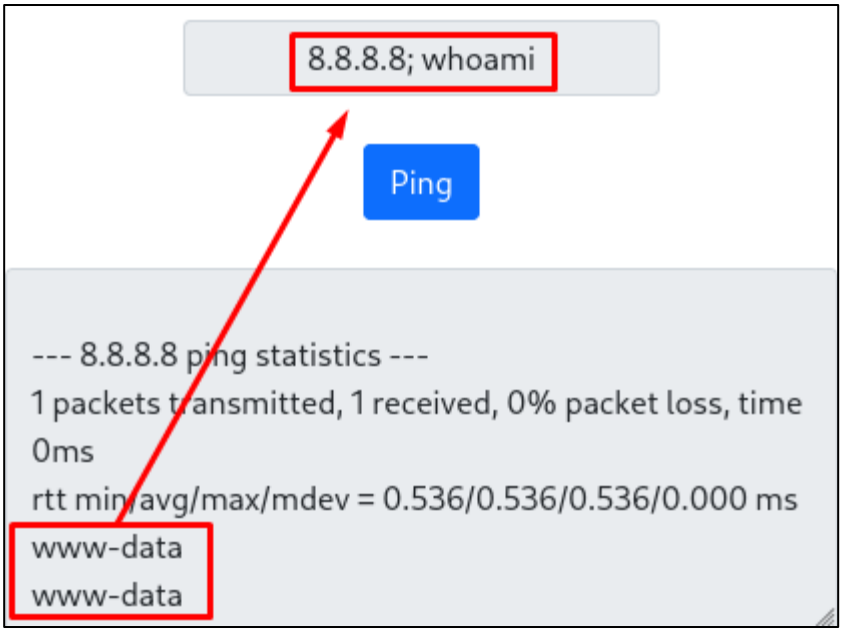
In the following website, a “PING” function could be manipulated to execute commands on the system by using a separator “;” which tricks the function to execute a new command after the separator, essentially allowing an attacker who have access to the admin panel to execute any command on the internal server.

Vulnerability Details

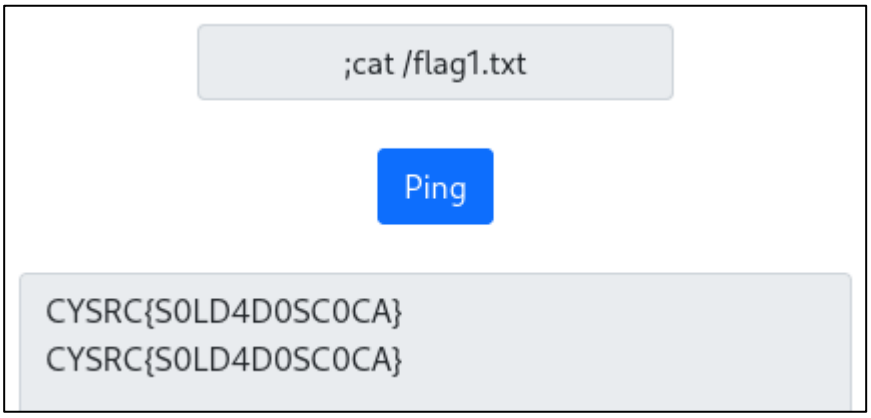
During the audit, after gaining credentials and access to the admin panel under “panel.php”, the following page is presented, and it seems like it executes a “PING” command:



As this command being executed on the server, it's possible to use a separator that allows to "chain" commands to a single line. The command that's being run on the server looks something like the following: "ping 8.8.8.8; whoami"



After some exploration, another flag was found: CYSRC{S0LD4D0SC0CA}



Mitigation

- **Filter the input:** As this option PING an IP, the best practice is to allow only numbers and dots to be inserted into the input. This will prevent an attacker from abusing this feature.
- **Don't allow command chaining:** Disallowing command chaining will also prevent this vulnerability from being exploited.

2 Known Vulnerabilities

Severity | Critical **Probability | Critical**

Vulnerability Description

Known Vulnerabilities refers to publicly disclosed security flaws in software or hardware that are documented in databases like the **CVE (Common Vulnerabilities and Exposures)**.

In the following webserver, multiple Known Vulnerabilities may be exploited due to outdated software and tools.

Vulnerability Details

PHP 8.2.7 Known Vulnerabilities (and potential RCE CVE):

https://www.cvedetails.com/version/1658960/PHP-PHP-8.2.7.html

https://www.exploit-db.com/exploits/52047

Apache 2.4.38 Known Vulnerabilities:

https://www.cvedetails.com/version/613554/Apache-Http-Server-2.4.38.html

Mitigation

- **Updates the patches:** It's important to update the system, services and tools on a regular basis under a program as attackers find new vulnerabilities regularly.

3 SSRF

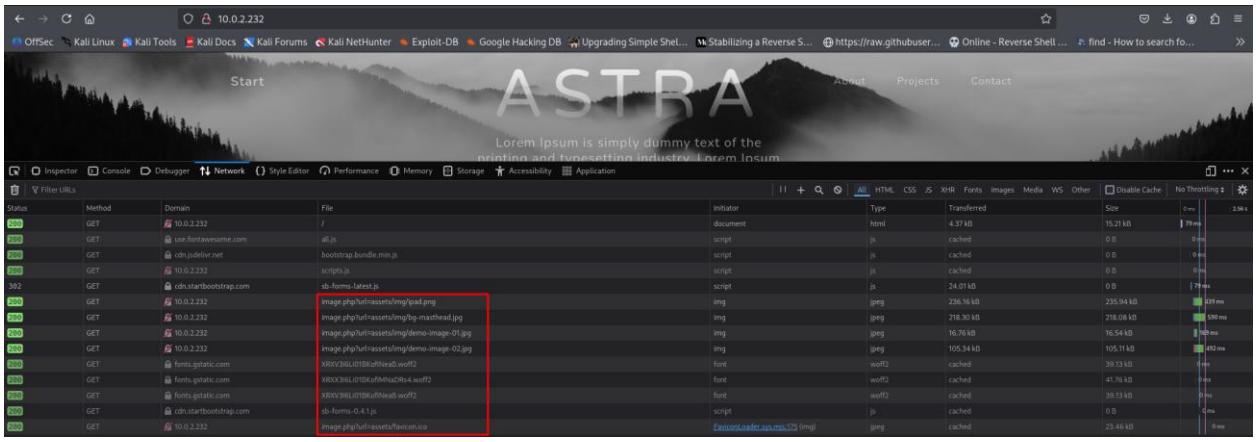
Severity | High **Probability | High**

Vulnerability Description

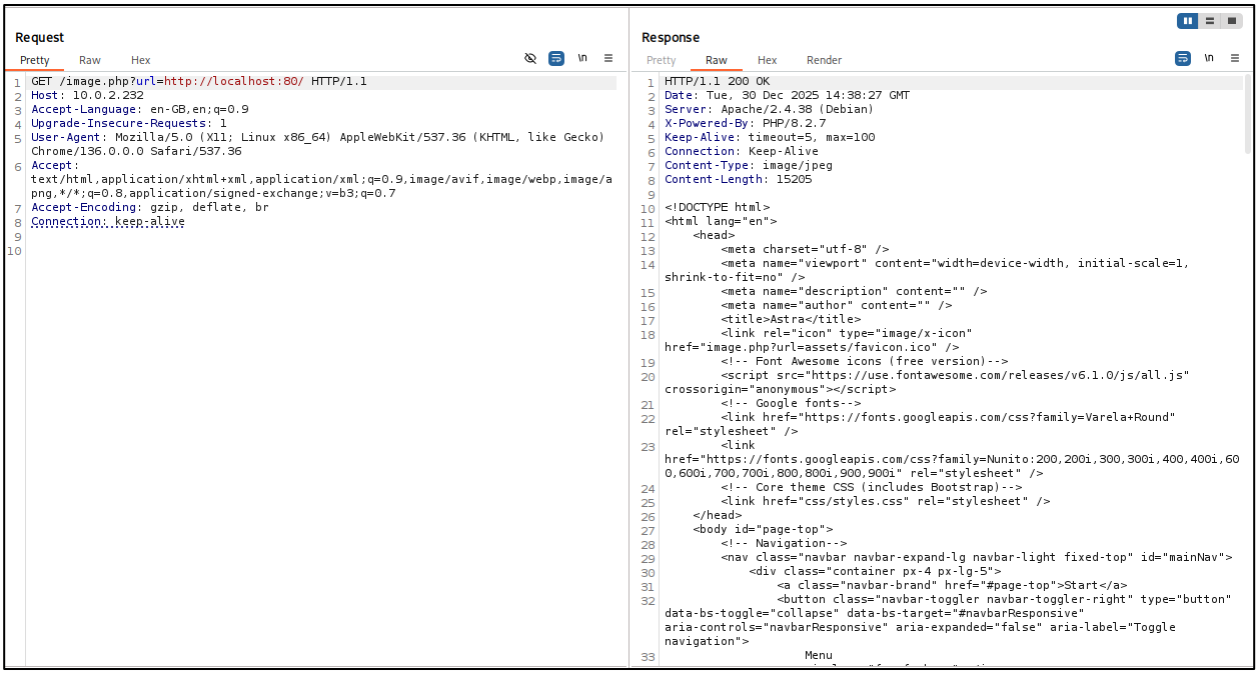
Server-Side Request Forgery (SSRF) is a vulnerability that allows an attacker to force a server-side application to initiate HTTP requests to arbitrary internal or external resources. This can enable attackers to access internal services, bypass network access controls, interact with internal systems, and retrieve sensitive information that is not directly exposed to the internet.

Vulnerability Details

During the audit, I found that the server includes image files on the website:



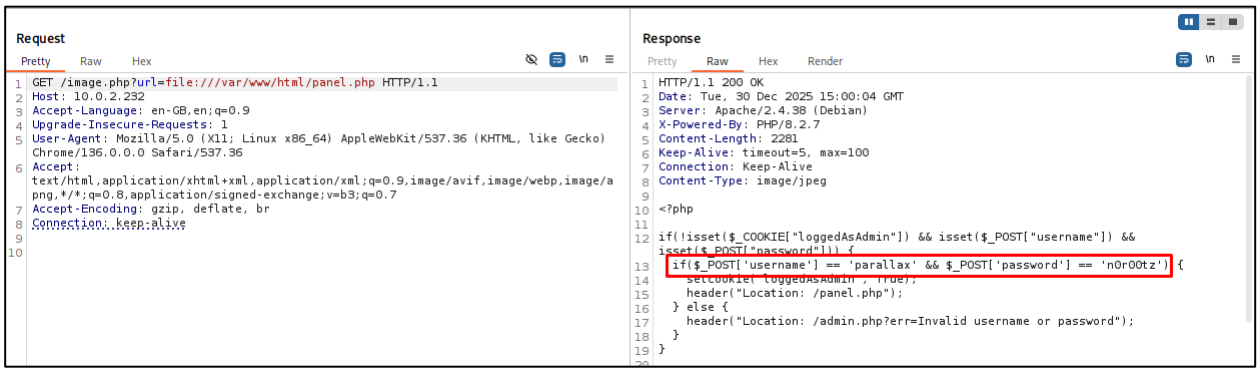
The request can be intercepted and manipulated by trying to access the backend server via “localhost”. As the image shows, by trying to access http://localhost:80 with burp, I indeed get content that is sent to the backend server, confirming SSRF is possible.



After some trial and error, using the following payload it’s possible to access files on the backend server. In the following example I’ve accessed the file “/etc/passwd”:

Inside the file “/var/www/html/panel.php”, credentials were found allowing access to the admin panel. “parallax: n0r00tz”

Also, a flag was found at the bottom of the page: “#flag2 CYSRC{1MH4X0R8888}”



Mitigation

- **Never trust user input:** Prevent opening or accessing files by the URL parameter without filters and sanitation, as the parameters can be manipulated by an attacker.
- **Implement a Whitelist/Blacklist:** Allow only trusted sources and domains to access the system, and block domains that don’t need to have access to the internal system.
- **Limit the ports who can access the system:** Preventing protocols like file:// will limit this attack vector.
- **Implement a firewall:** Implementing a firewall will prevent user’s from entering certain characters which can help reduce the risk of exploitation.

4 XSS

Severity | High **Probability | High**

Vulnerability Description

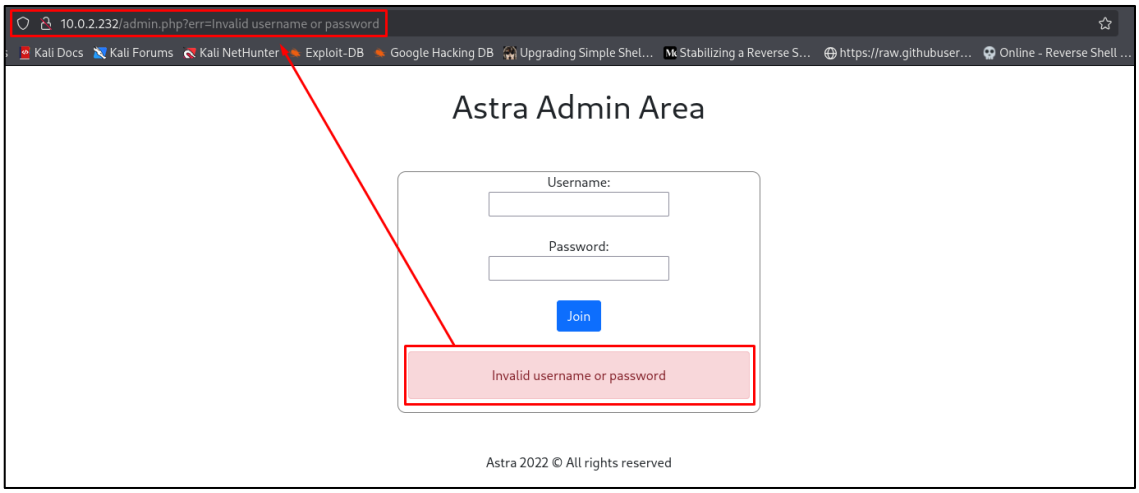
Cross-Site Scripting (XSS) is a client-side injection vulnerability where an attacker injects malicious script into a trusted website and force an action to execute on another user's browser.

In the following website, XSS was possible due to the web server reflecting the error message in the URL to the web page via the "incorrect credentials" message. This vulnerability makes it possible to inject a malicious payload into the URL and execute actions on another user's browser when accessing the login form via the malicious link under a trusted domain.

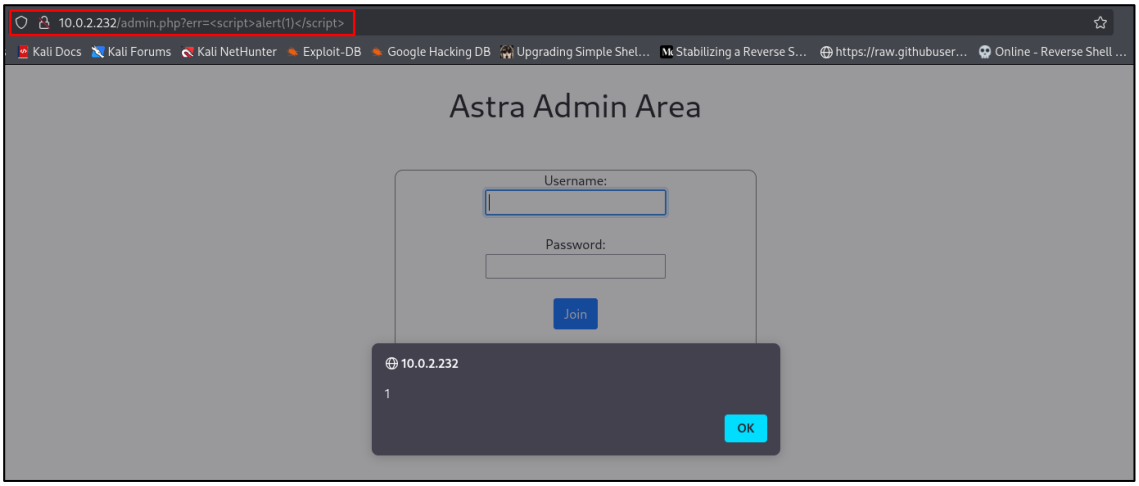
Vulnerability Details

During the test, I have found XSS in the login form in the path "/admin".

By entering wrong credentials to the form, the error message is reflected from the URL.



By changing the parameter to an XSS alert payload, the script runs on the browser, allowing an attacker to manipulate actions on a victim's browser, and executing JS actions by sending a malicious URL.



Mitigation

- **HTML Escaping:** Implement HTML Escaping which encode dangerous characters (for example < > ” ’ ;). This will prevent those characters from being used for unwanted actions on the web server.
- **Implement a Strict CSP:** This is a powerful "safety net" that tells the browser which sources of scripts are trusted.
- **Never Trust User Input:** Treat all incoming data as untrusted, including URL parameters, headers, and even data from internal users.
- **Input Validation:** Use "whitelist" validation to ensure data matches expected formats.
- **Implement a firewall:** Implementing a firewall will prevent users from entering certain characters which can help reduce the risk of exploitation.

5 Information Disclosure

Severity | Medium **Probability | Medium**

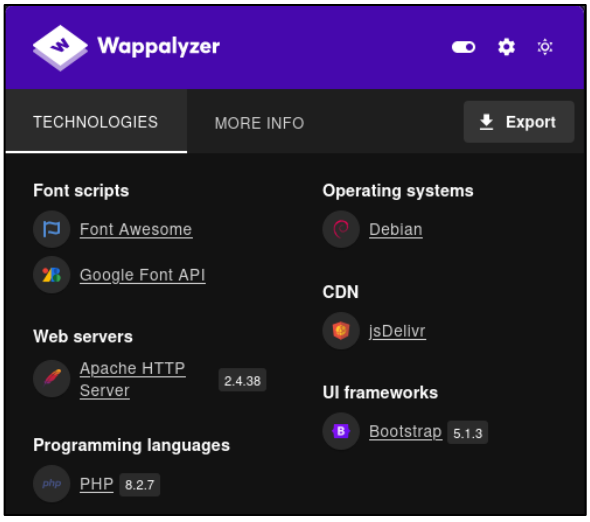
Vulnerability Description

An Information Disclosure vulnerability is a flaw where a system unintentionally reveals sensitive data to unauthorized users, often due to misconfigurations, poor error handling, or insecure design, giving attackers insights to launch more effective attacks, leading to data breaches or further exploitation

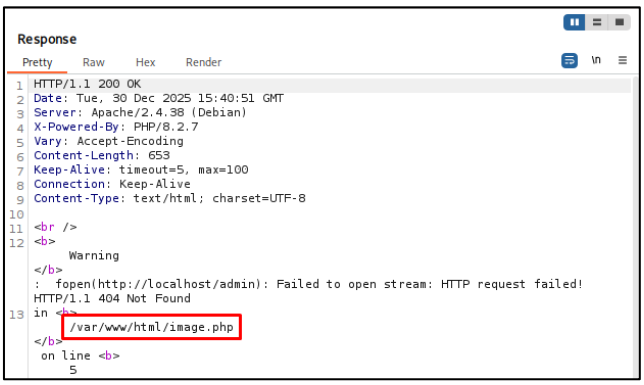
Vulnerability Details

During the audit, I've found information that can be used for attacking the system. The information gathered contained:

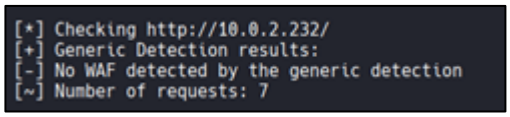
- Web server (Apache 2.4.38)
- Programming Language (PHP 8.2.7)
- UI Framework (Bootstrap 5.1.3)
- Operating System (Debian)



- Web server paths (/var/www/html/ AND “panel.php”(



- No usage of WAF



Mitigation

- **Secure API Responses:** Only return the minimum data fields necessary for a request. avoid exposing sensitive files in the response or request.
- **Encryption:** Encrypt sensitive data even if isn’t accessible by users, as this ensures the data is protected if an attack occurs.

Finding Classification

Severity

The finding's severity relates to the impact which might be inflicted to the organization due to that finding. The severity level can be one of the following options, and is determined by the specific attack scenario:

Critical – Critical level findings are ones which may cause significant business damage to the organization, such as:

- Significant data leakage
- Denial of Service to essential systems
- Gaining control of the organization's resources (For example Servers, Routers, etc.)

High – High level findings are ones which may cause damage to the organization, such as:

- Data leakage
- Execution of unauthorized actions
- Insecure communication
- Denial of Service
- Bypassing security mechanisms
- Inflicting various business damage

Medium – Medium level findings are ones which may increase the probability of carrying out attacks, or perform a small amount of damage to the organization, such as –

- Discoveries which makes it easier to conduct other attacks
- Findings which may increase the amount of damage which an attacker can inflict, once he carries out a successful attack
- Findings which may inflict a low level of damage to the organization

Low – Low level findings are ones which may inflict a marginal cost to the organization, or assist the attacker when performing an attack, such as –

- Providing the attacker with valuable information to help plan the attack
- Findings which may inflict marginal damage to the organization
- Results which may slightly help the attacker when carrying out an attack, or remaining undetected

Informative – Informative findings are findings without any information security impact. However, they are still brought to the attention of the organization.

Flags found

Flag ID	Flag content	Where it was found
Flag 1	CYSRC{S0LD4D0SC0CA}	Inside the internal server, under “/” directory.
Flag 2	CYSRC{1MH4X0R8888}	Inside the “panel.php” file.